



# Documento de implementación

**Sistema:** Tiendita Maday

**Cliente:** La Familia

**Versión:** 2.0

**Fecha:** 16 de julio de 2026

**Revisión analizada:** 023340e

Inventario técnico de frameworks, lenguajes, librerías, base de datos, APIs, servicios y decisiones de construcción.  
Las versiones se obtuvieron de los manifiestos del repositorio.

# 1. Resumen tecnológico

| Capa               | Implementación      | Versión / referencia                     |
|--------------------|---------------------|------------------------------------------|
| Interfaz           | Angular             | 21.2                                     |
| Lenguaje frontend  | TypeScript          | 5.9                                      |
| Componentes UI     | PrimeNG             | 21.1                                     |
| Reactividad        | RxJS                | 7.8                                      |
| Pruebas frontend   | Vitest + jsdom      | 4.0 / 28; suite aún no ejecutable        |
| API                | Node.js + Express   | Express 4.18                             |
| Lenguaje backend   | JavaScript CommonJS | Node compatible                          |
| Base de datos      | PostgreSQL          | Imagen Docker/configuración administrada |
| Driver             | node-postgres (pg)  | 8.11                                     |
| Especificación API | OpenAPI             | Generada en <code>src/openapi.js</code>  |
| Despliegue         | Docker / Railway    | Configuración versionada                 |

Para desarrollo y CI se recomienda **Node.js 24 LTS** o una versión admitida por Angular 21.2, fijada de forma idéntica en todos los entornos. Evitar versiones impares o etiquetas `latest` en producción.

## 2. Organización del repositorio

| Ruta                   | Contenido                                                           |
|------------------------|---------------------------------------------------------------------|
| Arquitectura/FrontEnd/ | Aplicación Angular, assets, configuración, pruebas y builds.        |
| Arquitectura/Backend/  | API Express, rutas, middleware, controladores, modelos y tests.     |
| Arquitectura/Database/ | Esquema Mermaid, <code>init.sql</code> y migraciones.               |
| Entregables/           | Documentación final en PDF/PPTX.                                    |
| Entregables/Fuentes/   | Markdown, capturas y generador reproducible.                        |
| Metodologia/           | Antecedentes históricos; no siempre reflejan la versión actual.     |
| Raíz                   | Compose, Dockerfiles Railway, configuración y guía del repositorio. |

### 2.1 Frontend

La aplicación se organiza por características bajo `src/app/features`: autenticación, inicio, producto, carrito, checkout, pedidos, perfil, caja, inventario y administración. Los elementos compartidos residen en `guards`, `servicios` y componentes reutilizables.

### 2.2 Backend

El backend separa `routes`, `middleware`, `controllers`, `models`, `services`, `config` y `utils`. `src/index.js` compone la aplicación y `src/routes/index.js` asigna políticas por dominio.

## 3. Framework frontend

Angular fue elegido por su enrutamiento, inyección de dependencias, formularios, compilación y carga diferida. La revisión actual usa componentes modernos y rutas lazy para evitar cargar todas las áreas al inicio.

### 3.1 Paquetes de ejecución

| Librería                                           | Uso                                                    |
|----------------------------------------------------|--------------------------------------------------------|
| @angular/core/common/forms/router/platform-browser | Núcleo, plantillas, formularios, navegador y rutas.    |
| @angular/service-worker                            | Capacidades PWA y actualización de recursos.           |
| primeng                                            | Componentes de interfaz complementarios.               |
| rxjs                                               | Flujos asíncronos y HTTP.                              |
| leaflet                                            | Mapas, ubicación y zonas de entrega.                   |
| flatpickr                                          | Selección y presentación de fechas.                    |
| jspdf                                              | Comprobantes o documentos PDF desde la interfaz.       |
| @emailjs/browser                                   | Integración histórica/cliente de correo donde aplique. |

### 3.2 Herramientas de desarrollo

| Herramienta         | Uso                                         |
|---------------------|---------------------------------------------|
| Angular CLI / build | Servidor local, compilación y optimización. |
| TypeScript          | Tipado estático y compilación.              |
| Vitest              | Motor de pruebas configurado por Angular.   |
| jsdom               | DOM simulado para tests.                    |
| Prettier            | Formato consistente.                        |

### 3.3 Comandos

| Comando               | Propósito                                                              |
|-----------------------|------------------------------------------------------------------------|
| npm start             | Ejecutar ng serve en desarrollo.                                       |
| npm run build         | Compilar la aplicación.                                                |
| npm run build:railway | Generar entorno y compilar producción.                                 |
| npm test              | Ejecutar tests; actualmente requiere corregir descubrimiento de specs. |

## 4. Backend y reglas de negocio

Express 4 recibe solicitudes JSON, aplica CORS, monta rutas bajo /api, publica OpenAPI y delega los errores al middleware global.

## 4.1 Dependencias

| Librería                            | Función                                                      |
|-------------------------------------|--------------------------------------------------------------|
| express                             | API REST y middleware.                                       |
| pg                                  | Pool y consultas PostgreSQL.                                 |
| dotenv                              | Variables locales.                                           |
| cors                                | Política de origen.                                          |
| bcryptjs                            | Hash y verificación de contraseñas.                          |
| jsonwebtoken                        | Tokens de acceso y renovación.                               |
| google-auth-library                 | Validación del inicio de sesión Google.                      |
| cloudinary                          | Gestión de imágenes remotas.                                 |
| multer + multer-storage-cloudinary  | Recepción y almacenamiento de cargas.                        |
| @emailjs/nodejs, nodemailer, resend | Alternativas/configuración de correo presentes en el código. |

La presencia de varias bibliotecas de correo refleja evolución del proyecto. Antes de producción se debe confirmar un solo proveedor activo, retirar rutas no usadas y rotar credenciales.

## 4.2 Comandos

| Comando              | Propósito                              |
|----------------------|----------------------------------------|
| npm run dev          | Ejecutar con reinicio automático.      |
| npm start            | Aplicar migraciones y levantar la API. |
| npm run migrate      | Ejecutar migraciones pendientes.       |
| npm run seed:catalog | Cargar catálogo inicial controlado.    |
| npm test             | Ejecutar 21 pruebas registradas.       |

## 5. Gestor de base de datos

PostgreSQL concentra información de identidad, catálogo, existencias, ventas, abasto, caja, finanzas y auditoría. El diagrama lógico documenta 33 entidades; el esquema ejecutable y migraciones pueden contener tablas auxiliares adicionales.

### 5.1 Archivos

| Archivo                    | Función                                                           |
|----------------------------|-------------------------------------------------------------------|
| Database/init.sql          | Instalación base para una base nueva.                             |
| Database/schema.mmd        | Diagrama entidad-relación legible.                                |
| Database/migrations/*.sql  | Cambios incrementales y compatibles con instalaciones existentes. |
| Backend/scripts/migrate.js | Ordena, registra y ejecuta migraciones no aplicadas.              |

## 5.2 Criterios de persistencia

- UUID para identificar la mayoría de registros.
- Llaves foráneas para conservar relaciones.
- Transacciones en ventas, caja y cambios compuestos.
- Historial separado para datos auditables.
- Stock modificado en servidor, nunca confiando en un total del navegador.
- RespalDOS lógicos compatibles con `pg_dump` y `pg_restore`.

## 6. APIs internas

La API expone 82 rutas y 138 operaciones documentadas en la revisión. Los clientes consumen JSON bajo `/api`.

| Categoría      | Ejemplos de recursos                                              |
|----------------|-------------------------------------------------------------------|
| Identidad      | auth, users, employees, sessions.                                 |
| Catálogo       | categories, units, inventory, barcodes, price history, reviews.   |
| Operación      | purchases, promotions, cash register, inventory movements.        |
| Abasto         | suppliers, purchase orders, stock receipts, expiration batches.   |
| Administración | schedules, customers, notifications, shop config, delivery zones. |
| Finanzas       | expenses, till movements, cash audit.                             |

OpenAPI está disponible como JSON en `/swagger.json` y como interfaz en `/api-docs`. La documentación histórica `Metodologia/openapi.yaml` puede quedar rezagada; la fuente vigente es `Backend/src/openapi.js`.

## 7. Servicios externos

| Servicio     | Implementación                  | Variables principales                                 |
|--------------|---------------------------------|-------------------------------------------------------|
| PostgreSQL   | pg                              | DATABASE_URL                                          |
| Google OAuth | google-auth-library y frontend  | GOOGLE_CLIENT_ID                                      |
| PayPal       | llamadas REST desde backend     | PAYPAL_CLIENT_ID, PAYPAL_SECRET, base y moneda        |
| Cloudinary   | SDK + Multer                    | CLOUDINARY_URL o credenciales equivalentes            |
| Correo       | EmailJS / proveedor configurado | EMAILJS_* o variables del proveedor                   |
| Mapas        | Leaflet / OpenStreetMap         | Sin secreto para teselas públicas; respetar políticas |
| Railway      | servicios y variables           | PORT, URLs y secretos de cada entorno                 |

### 7.1 Política de secretos

- Versionar nombres y ejemplos, nunca valores reales.
- Usar secretos independientes para desarrollo, pruebas y producción.
- Rotar secretos al transferir el proyecto.

- Restringir permisos de cuentas de terceros.
- No copiar tokens en capturas, PDFs, logs o tickets.

## 8. Configuración por entorno

| Variable                           | Frontend                              | Backend             | Observación                              |
|------------------------------------|---------------------------------------|---------------------|------------------------------------------|
| API_BASE_URL                       | Sí, al compilar                       | No                  | Debe terminar en /api.                   |
| FRONTEND_URL                       | No                                    | Sí                  | Origen permitido y enlaces.              |
| BACKEND_URL                        | No                                    | Sí                  | Servidor publicado en documentación.     |
| DATABASE_URL                       | No                                    | Sí                  | Secreto de conexión.                     |
| JWT_SECRET /<br>JWT_REFRESH_SECRET | No                                    | Sí                  | Obligatorios y aleatorios en producción. |
| PORT                               | Runtime                               | Runtime             | Asignado por plataforma.                 |
| Credenciales de terceros           | Solo identificadores públicos mínimos | Secretos en backend | Separar sandbox/live.                    |

## 9. Construcción y despliegue

### 9.1 Desarrollo local

`docker-compose.yml` levanta frontend, backend y PostgreSQL con volúmenes. El backend aplica migraciones antes de iniciar y la base escucha en el puerto de host 5433 para evitar colisiones comunes.

### 9.2 Producción

Los Dockerfiles de raíz existen para evitar problemas de rutas con espacios y paréntesis en Railway. El frontend compila en una etapa y se sirve como contenido estático; el backend instala dependencias de producción y expone la API.

### 9.3 Riesgos de reproducibilidad observados

| Hallazgo                                              | Riesgo                              | Mejora                                                   |
|-------------------------------------------------------|-------------------------------------|----------------------------------------------------------|
| <code>postgres:latest</code> en Compose               | Actualización mayor inesperada      | Fijar versión mayor y probar migración.                  |
| <code>node:latest</code> en Dockerfile frontend local | Build diferente entre fechas        | Fijar Node 24 LTS o versión aprobada.                    |
| Variantes de Dockerfile                               | Despliegue desde archivo equivocado | Documentar un camino canónico y retirar duplicados.      |
| Node 20 genérico en Railway                           | Cambios dentro de la rama 20        | Fijar parche/digest compatible con Angular.              |
| Varias librerías de correo                            | Configuración ambigua               | Conservar un proveedor activo.                           |
| Root Dockerfile backend no muestra migraciones        | Esquema podría quedar atrás         | Verificar comando Railway y copiar/ejecutar migraciones. |

## 10. Calidad y pruebas

| Área               | Estado implementado                                                   |
|--------------------|-----------------------------------------------------------------------|
| Backend            | Node test runner; 19 aprobadas y 2 omitidas sin base aislada.         |
| Frontend           | Vitest configurado, pero el comando no descubre una suite ejecutable. |
| Compilación        | Angular produce bundle con advertencias registradas.                  |
| Integración manual | Cinco roles recorridos con datos locales y capturas.                  |
| API                | Contrato OpenAPI generado desde código.                               |
| Base               | Migraciones versionadas; restauración de respaldo pendiente.          |

El detalle de casos, resultados, incidencias y correcciones se conserva en `03_Reporte_de_Pruebas_y_Limitaciones.pdf`.

## 11. Convenciones de mantenimiento

- Crear ramas pequeñas y commits con propósito identificable.
- No mezclar migraciones destructivas con cambios no probados.
- Agregar/actualizar OpenAPI cuando cambia un endpoint.
- Proteger rutas en backend aunque exista un guard frontend.
- Usar tokens visuales compartidos antes de agregar colores o espacios ad hoc.
- Ejecutar build, tests y prueba de humo antes de desplegar.
- Registrar commit, variables modificadas, migraciones y resultado posterior.
- Actualizar documentos que se vuelvan falsos con el cambio.

## 12. Criterio de implementación completa

Una función se considera implementada cuando:

1. Existe navegación o endpoint accesible al rol correcto.
2. Valida entradas y maneja errores previsibles.
3. Persiste o consulta información de forma consistente.
4. No confía en datos sensibles calculados por el cliente.
5. Tiene evidencia de prueba proporcional al riesgo.
6. Está documentada en API/manual cuando afecta a otros.
7. Puede configurarse sin publicar secretos.
8. Se despliega y puede revertirse con un procedimiento conocido.

Este documento no sustituye los manifiestos ni el código. Su función es explicar cómo se implementa la revisión y señalar decisiones que deben estabilizarse antes de producción.